Practica 3

1. ¿Qué es el Shell Scripting? ¿A qué tipos de tareas están orientados los script? ¿Los scripts deben compilarse? ¿Por qué?

La shell es un interprete de comandos interactivo y configurable en sistemas operativos de tipo *nix que provee estructuras de control para programar. El shell scripting consiste en escribir un conjunto de comandos e instrucciones dentro de un archivo para que la shell los interprete y ejecute de manera secuencial
Los scripts no deben compilarse, se ejecutan de forma interpretada. La shell lee e interpreta las intrucciones linea por linea. Para ejecutarlos solo hace falta indicar que interprete a utilizar y otorgar permisos de ejecucion

2. Investigar la funcionalidad de los comandos echo y read. a. ¿Cómo se indican los comentarios dentro de un script? b. ¿Cómo se declaran y se hace referencia a variables dentro de un script?

echo: Se utiliza para imprimir texto por la salida estandar

read: Se utiliza para leer una linea desde la entrada estandar (teclado) y guardar ese valor en una variable. Ej: read var1

a. Los comentarios se indican iniciando la linea o instruccion con el simbolo #

Excepcion / Shebang: Si la primera linea del archivo comienza especificamente con #!, esta linea es interpretada como la ruta del interprete a utilizar para ejecutar el script (#!/bin/bash)

b. Declaracion y creacion:
Se declara asignando un valor con el signo =, sin espacios alrededor de este. Ej
NOMBRE=”ejemplo”
Los nombres son case sensitive y no pueden empezar con un numero
Bash soporta tanto cadenas de texto como arrays
Por defecto las variables son globales: para limitar el alcance de una variable al cuerpo de una funcion se declara usando la palabra clave global

Referencia y acceso:
Para acceder al valor almacenado en una variable, se antepone el simbolo $ a su nombre. Ej $nombre
Para evitar ambiguedades al concatenar el nombre de la variable con otro texto se usan llaves. Ej ${nombre}texto
En el caso de los arrays, las llaves son obligatorias para acceder a una posicion especifica. Ej ${array}

3. Crear dentro del directorio personal del usuario logueado un directorio llamado

practica-shell-script y dentro de él un archivo llamado mostrar.sh cuyo contenido

sea el siguiente:

#!/bin/bash

Comentarios acerca de lo que hace el script

Siempre comento mis scripts, si no lo hago hoy,

mañana ya no me acuerdo de lo que quise hacer

echo "Introduzca su nombre y apellido:"

read nombre apellido

echo "Fecha y hora actual:"

date

echo "Su apellido y nombre es:"

echo "$apellido $nombre"

echo "Su usuario es: `whoami`"

echo "Su directorio actual es:"

a. Asignar al archivo creado los permisos necesarios de manera que pueda

ejecutarlo

b. Ejecutar el archivo creado de la siguiente manera: ./mostrar

c. ¿Qué resultado visualiza?

d. Las backquotes (`) entre el comando whoami ilustran el uso de la sustitución de comandos. ¿Qué significa esto?

e. Realizar modificaciones al script anteriormente creado de manera de poder mostrar distintos resultados (cuál es su directorio personal, el contenido de un directorio en particular, el espacio libre en disco, etc.). Pida que se introduzcan por teclado (entrada estándar) otros datos.

a. chmod +x mostrar.sh (tambien se puede hacer chmod 755 mostrar.sh)

b y c. bash:mostrar: No existe el archivo (falta el .sh)

d. El uso de comillas invertidas (`comando`) o de la sintaxis \$(comando) implementa el reemplazo / sustitución de comandos

Esto significa que la shell ejecuta el comando encerrado en segundo plano y reemplaza esa expresión por el texto resultante de su salida estándar dentro de la cadena. En el ejemplo, `whoami` se ejecuta y se sustituye directamente por el nombre del usuario logueado

e. #!/bin/bash

Script mostrar.sh modificado con más funcionalidades
echo "Introduzca su nombre y apellido:"
read nombre apellido
echo "Fecha y hora actual:" date
echo "Su apellido y nombre es:"
echo "$apellido $nombre"
echo "Su usuario es: $(whoami)"
Se agrega el comando pwd para mostrar el directorio actual
echo "Su directorio actual es:"
pwd
Mostrar el directorio personal del usuario echo "Su directorio personal (HOME) es:"
echo "$HOME"
echo "--"
Solicitar un directorio para listar su contenido echo "Ingrese la ruta de un directorio para ver su contenido (ejemplo: /tmp o /var):"
read dir_usuario
echo "Contenido del directorio $dir_usuario:" ls -l "$dir_usuario"
echo "--"
Mostrar el espacio libre en disco
echo "Espacio libre en disco:" df -h

4. Parametrización: ¿Cómo se acceden a los parámetros enviados al script al momento de su invocación? ¿Qué información contienen las variables $#, $*, $? y $HOME dentro de un script?

Acceso a parametros de invocacion:
Se acceden mediante variables posicionales $1, $2, $3, $4, $N. La variables $1 contiene el primer argumento, $2 el segundo y asi sucesivamente. Ademas $0 almacena el nombre o comando de invocacion del script

Variables especiales $# cantidad de argumentos
$* Lista completa de todos los argumentos pasados al script
$? Almacena el valor de retorno del ultimo comando o funcion ejecutada
$HOME Es una variable de entorno que contiene la ruta al directorio personal del usuario actual

5. ¿Cual es la funcionalidad de comando exit? ¿Qué valores recibe como parámetro y cuál es su significado?
Causa la terminacion o finalizacion del script

Parametros y significado:
Recibe un valor numerico entre 0 y 255
0 significa que el script finalizo exitosamente
1 a 255 indican un codigo de error

6. El comando expr permite la evaluación de expresiones. Su sintaxis es: expr arg1 op arg2, donde arg1 y arg2 representan argumentos y op la operación de la expresión. Investigar que tipo de operaciones se pueden utilizar.
Se usa para evaluar expresiones. Entre las operaciones que permite utilizar se encuentran:
Aritmeticas: Suma +, resta -, multiplicacion *, division / y modulo %

Relacionales / comparacion: igualdad =, desigualdad !=, menor que <, mayor que >, mayor o igual >=. Estos devuelven 1 si la condicion es veradera o 0 si es falsa.

Operaciones sobre cadenas: Comparacion de patrones: match, calculo de longitud de cadena: length, extraccion de subcadenas: substr o busqueda de posicion: index

7. El comando test expresion permite evaluar expresiones y generar un valor de retorno, true o false. Este comando puede ser reemplazado por el uso de corchetes de la siguiente manera [expresión]. Investigar qué tipo de expresiones pueden ser usadas con el comando test. Tenga en cuenta operaciones para: evaluación de archivos, evaluación de cadenas de caracteres y evaluaciones numéricas.

El comando test o la sintaxis equivalente [expresion] evalua condiciones logicas:
Evaluacion de archivos:
-f archivo: verdadero si el archivo existe y es un archivo regular
-d archivo: verdadero si existe y es un directorio
-e archivo: verdadero si el archivo existe
-r -w -x archivo: verdaros si el archivo existe y posee permisos de lectura, escritura o ejecucion
-s archivo: verdadero si existe y su tamano es mayor a 0
-L archivo: verdadero si es un enlace simbolico

Evaluacion de cadenas
“$cad1” = “$cad2” (o ==) igualdad de cadenas
“$cad1” != “$cad2” desigualdad de cadenas
-z “$cadena” verdadero si la longitud de la cadena es 0 (vacia)
-n “$cadena” verdadero si longitud mayor que 0

Evaluaciones numericas
$a -eq $b: igual
$a -ne $b distinto
$a -gt $b mayor que
$a -ge $b mayor o igual
$a -lt $b menor que
$a -le $b menor o igual

8. Estructuras de control. Investigue la sintaxis de las siguientes estructuras de control incluidas en shell

scripting: ➢ if ➢ case ➢ while ➢ for ➢ select

```
if [ condicion ]; then
# bloque elif [ condicion ]; then
# bloque else
# bloque
fi

case $variable in "valor1")
# bloque
;;
"valor2")
# bloque
;;
*)
# bloque por defecto ;;
esac

while [condicion]; do
#bloque
done

for ((i=0; i < N; i++)); do
# bloque
done
# Con lista de valores (foreach):
for var in valor1 valor2 valorN; do
# bloque
done

select variable in opcion1 opcion2 opcion3; do
# el valor elegido queda guardado en $variable
# bloque
done
```

9. ¿Qué acciones realizan las sentencias break y continue dentro de un bucle? ¿Qué parámetros reciben?
break [n]: Interrumpe y corta la ejecución de n niveles de bucles. Si no se pasa el parámetro n, corta por defecto el bucle actual ($n=1$).

continue [n]: Salta el resto de la iteración actual e inicia la siguiente iteración del n-ésimo bucle que la contiene

10. ¿Qué tipo de variables existen? ¿Es shell script fuertemente tipado? ¿Se pueden definir arreglos? ¿Cómo?

Tipos de variables: *Bash* maneja principalmente cadenas de texto (strings) y arreglos (arrays).

Según su alcance, pueden ser locales (declaradas con `local` dentro de funciones) o globales, y variables de entorno (que se heredan a procesos hijos si se exportan con `export`).

- ¿Es fuertemente tipado?: No. Es de tipado débil / no tipado; las variables son tratadas conceptualmente como cadenas de texto, y se interpretan como números según el contexto de evaluación (por ejemplo, con `let` o expresiones matemáticas).
- Definición de arreglos: Sí, se pueden definir arreglos.
- Creación: Vacío `arreglo=()` o inicializado `arreglo=(1 2 3 "hola")`.
- Asignación concreta: `arreglo="spam"`.
- Acceso: `${arreglo}` (las llaves son obligatorias).
- Todos los elementos: `${arreglo[@]}` o `${arreglo[*]}`.
- Longitud del arreglo: `${#arreglo[@]}`.
- Borrado de un elemento: `unset arreglo`.

11. ¿Pueden definirse funciones dentro de un script? ¿Cómo? ¿Cómo se maneja el pasaje de parámetros de una función a la otra?

Hay dos formas de hacerlo
function nombre {bloque}
nombre() {bloque}

Pasaje de parametros:
Se pasan al invocar la funcion de la misma forma que a un script. Ej: funcion arg1 arg2

Dentro de la funcion, los argumentos se leen con las variables posicionales $1 $2 $3 etc.

Para retornar un codigo numerico de estado se utiliza la instruccion return, cuyo valor puede consultarse inmediatamente despues con $?

12. Evaluación de expresiones. a. Realizar un script que le solicite al usuario 2 números, los lea de la entrada Standard e imprima la multiplicación, suma, resta y cual es el mayor de los números leídos. b. Modificar el script creado en el inciso anterior para que los números sean recibidos como parámetros. El script debe controlar que los dos parámetros sean enviados. c. Realizar una calculadora que ejecute las 4 operaciones básicas: +, - ,*, %. Esta calculadora debe funcionar recibiendo la operación y los números como parámetros

```
#!/bin/bash
# Script: ejercicio12a.sh
echo "Ingrese el primer número:"
read num1
echo "Ingrese el segundo número:"
read num2
# Operaciones aritméticas suma=$((num1 + num2)) resta=$((num1 - num2)) mult=$((num1 * num2))
echo "Suma: $suma"
echo "Resta: $resta"
echo "Multiplicación: $mult"
# Evaluación de cuál es el mayor
```

```
if [ $num1 -gt $num2 ]; then
echo "El número mayor es: $num1"
elif [ $num2 -gt $num1 ]; then
echo "El número mayor es: $num2" else
echo "Ambos números son iguales ($num1)"
fi
```

b.

```
#!/bin/bash
# Script: ejercicio12b.sh
# Control de parámetros recibidos
if [ $# -ne 2 ]; then
echo "Error: Debe enviar exactamente 2 números como parámetros."
echo "Uso: $0
```

c.

```
#!/bin/bash
# Script: calculadora.sh
if [ $# -ne 3 ]; then
echo "Error: Se esperan 3 parámetros."
echo "Uso: $0 num1=$1 operacion=$2 num2=$3
case $operacion in "+") res=$((num1 + num2))
echo "Resultado: $num1 + $num2 = $res" ;; "-") res=$((num1 - num2))
echo "Resultado: $num1 - $num2 = $res" ;; "*") res=$((num1 * num2))
echo "Resultado: $num1 * $num2 = $res" ;; "%")
if [ $num2 -eq 0 ]; then
echo "Error: División por cero no permitida."
exit 1
fi res=$((num1 % num2))
echo "Resultado: $num1 % $num2 = $res" ;; *)
echo "Operación inválida: '$operacion'. Operaciones válidas: +, -, * o %"
exit 1
;;
esac
```

13.

```
#!/bin/bash
# Script: cuadrados.sh
echo "Número | Cuadrado"
echo "-------------------" for ((i=1; i<=100; i++)); do
cuadrado=$((i * i))
echo "$i -> $cuadrado"
done
```

b.

```
#!/bin/bash
```

```
# Script: menu.sh PS3="Seleccione una opción (1-4): "
select opcion in "Listar" "DondeEstoy" "QuienEsta" "Salir"; do
case $opcion in "Listar")
echo "--- Contenido del directorio actual ---" ls -l ;; "DondeEstoy")
echo "--- Ruta del directorio actual ---"
pwd ;; "QuienEsta")
echo "--- Usuarios conectados al sistema ---"
who ;; "Salir")
echo "Saliendo del programa..."
break ;; *)
echo "Opción inválida. Intente nuevamente." ;;
esac
done
```

c.

```
#!/bin/bash # Script: verificar.sh if [ $# -ne 1 ]; then echo "Error: Ingrese el nombre de un archivo o
directorio." echo "Uso: $0 <nombre>”
exit 1
fi

nombre=$1
if [-e $”nombre”]; then

echo “El elemento ‘$nombre’ existe”
if [-d “$nombre”]; then
echo “tipo: Es un directorio”
elif [-f “$nombre”]; then
echo “Tipo: es un archivo regular”
else
echo “tipo: es otro tipo de archivo”
fi
else
echo “El elemento no existe”
echo “Creando directorio “$nombre”
mkdir “$nombre”
echo “Creado con exito”
fi
```

14.

```
#!/bin/bash
# Script: renombrar.sh

#validacion de parametros
if [$# -ne 3] then
echo “Parametros insuficientes”
echo “Uso : $0 <directorio> <-a|-b> <CADENA>”
exit 1
```

```
fi

dir=$1
opcion=$2
cadena=$3

#verificar si el directorio existe
if [! -d “$dir”]; then
echo “No existe o no es un directorio”
exit 1
fi

#verificar la opcion
if [“$opcion” != “-a”] && [“$opcion” != “-b”]; then
echo “Error opcion no valida”
exit 1
fi

#recorrer elementos dentro del dir
for archivo in “$dir”/*; do
#procesar unicamente si es un archivo regular
if [-f “$archivo”]; then
nombre =$(basename “$archivo”)
dir_padre=$(dirname “$archivo”)
case $opcion in
“-a”)
nuevo_nombre=”${nombre}${cadena}”
;;
“-b”)
nuevo_nombre=”${cadena}${nombre}”
;;
esac

mv “$archivo” “$dir_padre/$nuevo_nombre”
echo “Renombrado con exito”
fi
done
```

15.
El comando **cut** permite extraer secciones, columnas o campos delimitados de cada línea de un texto proveniente de un archivo o de la entrada estándar.

Principales parámetros:

- **-d "delimitador"** (o `--delimiter`): Especifica el carácter delimitador de campos (por defecto es *Tab*).

- **-f lista** (o `--fields`): Selecciona los campos o columnas específicos a extraer (ej. `-f1`, `-f1,3` o `-f1-4`).
- **-c lista** (o `--characters`): Extrae un rango exacto de caracteres por posición (ej. `-c 1-10`).
- **--complement**: Muestra todo el contenido **excepto** los campos elegidos.

Ejemplos de uso:

1. **Obtener la lista de nombres de usuarios del sistema desde /etc/passwd :** Utiliza `:` como delimitador y extrae la primera columna:

```
cut -d: -f1 /etc/passwd
```

1. **Extraer el usuario y su directorio HOME (campos 1 y 6):**

```
cut -d: -f1,6 /etc/passwd
```

1. **Extraer los primeros 5 caracteres de cada línea de un archivo:**

```
cut -c 1-5 archivo.txt
```

1. **Uso en tuberías (pipes) para procesamiento de datos:** Filtra los nombres de usuario y cuenta cuántos contienen la letra 'a':

```
cat /etc/passwd | cut -d: -f1 | grep a | wc -l
```

16. Realizar un script que reciba como parámetro una extensión y haga un reporte con 2 columnas, el nombre de usuario y la cantidad de archivos que posee con esa extensión. Se debe guardar el resultado en un archivo llamado reporte.txt
    #!/bin/bash
    #script: ejercicio16.sh

    if [$# -ne 1]; then

    echo “error: debe ingresar una extension como parametro”
    echo “uso $0 <extension>”
    echo “ejemplo: $0 txt”
    exit 1

fi

ext=$1
#limpiamos el punto iniciar si el usuario lo ingreso

```
ext=”${ext#.}”

archivo_reporte=”reporte.txt”

#titulo
echo “usuario cantidad archivos”
“$archivo_reporte”
echo “_____”
”$archivo_reporte”

#recorremos los usuarios en /etc/psswd

        cut -d: -f1 /etc/psswd | while read -r usuario; do
    home_dir=$(eval echo “-$usuario”2>/dev/null)
    if [-d “$home_dir”]; then
    cant=$(find “$home_dir” -user “$usuario” -name “*.$ext” -type f2V/dev/null | wc -1)
    if [“$cant” -gt 0]; then

        echo “$usuario $cant” >> “$archivo_reporte”
        fi

fi
done
```

17. Escribir un script que al ejecutarse imprima en pantalla los nombre de los archivos que se encuentran en el directorio actual, intercambiando minúsculas por mayúsculas, además de eliminar la letra a (mayúscula o minúscula). Por ejemplo, si en el directorio actual están los siguientes archivos:
➢ IsO
➢ pepE
➢ Maria y ejecutó: ./ejercicio17 , se obtendrá como resultado:
➢ iSo
➢ PEPe
➢ mRI Ayuda: Investigar el comando tr

```
#!/bin/bash

#script: ejercicio17.sh
#recorrer los elementos del dir actual
for archivo in *; do
if [-e “$archivo”]; then
  echo “$archivo” | tr -d ‘aA’ | tr ‘a-aA-Z’ ‘A-Za-z’
fi
done
```

18.Crear un script que verifique cada 10 segundos si un usuario se ha logueado en el sistema (el nombre del usuario será pasado por parámetro). Cuando el usuario finalmente se loguee, el programa deberá mostrar el mensaje ”Usuario XXX logueado en el sistema” y salir.

```
#!/bin/bash
#script: ejercicio18.sh
#control de parametros
if [$# -ne 1]; then
echo “error: debe ingresar un nombre de usuario como parametro”
echo “uso: $0 <nombre_usuario>”
exit 1
fi

usuario=$1
#bucle
while true; do
#consultar
if who | grep -w “$usuario” > /dev/null; then
echo “usuario $usuario logueado en el sistema”
exit 0
fi
sleep 10
done
```

19. 19. Escribir un Programa de “Menu de Comandos Amigable con el Usuario” llamado

menú, el cual, al ser invocado, mostrará un menú con la selección para cada uno

de los scripts creados en esta práctica. Las instrucciones de cómo proceder

deben mostrarse junto con el menú. El menú deberá iniciarse y permanecer

activo hasta que se seleccione Salir. Por ejemplo:

MENU DE COMANDOS

03. Ejercicio 3

12. Evaluar Expresiones

13. Probar estructuras de control

...

Ingrese la opción a ejecutar: 03

```
#!/bin/bash
#script: menu.sh

while true; do
echo ”menu user friendly”
echo ”(3) ejercicio 3 mostrar datos del usuario”
echo “(12) evaluar expresiones, calculadora basica”
```

```
echo “(13) probar estructuras”
echo “(14) renombrar archivos”
echo “(16) reporte de archivos”
echo “(17) transformar nombres de archivos”
echo “(18) monitorear usuario”
echo “0 salir”
read -p “ingrese una opcion del menu”; opcion
case $opcion in

3 | “3”)
echo “./mostrar.sh”
12) read -p “ingrese el primer num” n1
read -p “ingrese la operacion” op
read -p “ingrese el segundo numero” n2
./calculadora.sh “$n1” “$op“ “$n2”
;;

13) ./verificar.sh
;;
esac
read -p
done
```

```
20. #!/bin/bash
#script: ejercicio20.sh
#arreglo vacio
pila=()
push(){
if [$# eq 0]; then
echo “error: proporcione un valor”
return 1
fi
pila+=(“$1”)
}

pop(){
local cantidad =${#pila[@]}
if [$cantidad -eq 0]; then
echo “pila vacia”
return 1
fi

local ultimo_i=$((cantidad -1))
local elemento=${pila[$ultimo_i]}
unset “pila[$ultimo_i]”
pila=(“${pila[@]}”)
echo “elemento removido: $elemento”
```

```
}

#funcion lentgh
lentgh (){
echo “longitud: ${pila[@]}”
}

#funcion print:
print(){
echo “elementos: ${pila[*]}”
}
```

21.

```
#!/bin/bash

# Script: ejercicio21.sh

num=(10 3 5 7 9 3 5 4)

productoria(){

   local resultado=1

   for n in "${num[@]}"; do

   resultado=$((resultado * n))

done

echo "La productoria de todos los elementos del arreglo es: $resultado"

}

# Invocación a la función

       productoria
```

22. Implemente un script que recorra un arreglo compuesto por números e imprima en pantalla sólo los números pares y que cuente sólo los números impares y los informe en pantalla al finalizar el recorrido.

```
 #!/bin/bash
#script: recorrido.sh
numeros=(10 11 12 13 14 15)
cant_impar=0
echo “numeros pares encontrados: ”
for n in “${numeros[@]}”; do
if [$((n%2)) -eq 0]; then
   echo “$n es par”
else
 cant_impar=$((cant_impar +1))
```

```
fi
done
echo “total de numeros impares: $cant_impar”
```

23.Complete este script de manera tal de implementar la suma elemento a elemento entre ambos vectores y que la misma sea impresa en pantalla de la siguiente manera:

```
#!/bin/bash
#script: ejercicio23.sh
vector1=(10 20 30 40)
vector2=(10 20 50 60)
longitud=${vector1[@]}
for (i=0; i<longitud; i++) do
sum=$((vector1[i] + vector2[i]))
echo “la suma de la posision $i es: $sum”
done
```

24. Realice un script que agregue en un arreglo todos los nombres de los usuarios del sistema pertenecientes al grupo “users”. Adicionalmente el script puede recibir como parámetro: ➢ “-b n”: Retorna el elemento de la posición n del arreglo si el mismo existe. Caso contrario, un mensaje de error. ➢ “-l”: Devuelve la longitud del arreglo ➢ “-i”: Imprime todos los elementos del arreglo en pantalla

```
#!/bin/bash
#script: ejercicio24.sh
usuarios=()
grupo_info=$(getent group users | cut -d: -f4)
IFS=’,’ read -ra lista <<< “$grupo_info”

for usr in “${lista_usr[@]}”; do
if [-n “$usr”]; then
usuario+=(”$usr”)
fi
done

case $1 in
“-b”)
pos=$2
if [-z “$pos”]; then
echo “error debe indicar la posiciona luego de -b”
exit 1
fi

if [“$pos” -ge 0] && [“$pos” -lt “${#usuarios[@]}”]; then
echo “elemento en la pos $pos: ${usuarios[$pos]}”
else
echo “pos invalida”
fi
```

```
;;
"-l")
echo "longitud del arreglo: ${usuarios[@]}"
;;
"*")
echo "elementos del arreglo: ${usuarios[*]}"
exit 1
;;
esac
```

25.
```
#!/bin/bash
#script: ejercicio25.sh
#revisar si al menos es 1
if [$# -lt 1]; then
echo "error parametros insuficientes (debe ser >1)"
exit 1
fi

inexistentes=0
posicion=1
parametros=("$0")

por param in ($parametros[@]); do
if [$posicion%2 -ne 0]; then
echo "evaluando posicion por ser impar"
if [-e $param]; then
echo "directorio existente"
else
echo "es otro tipo de archivo"
fi
else
echo"no existe en el sistema"
inexistentes=$(inexistentes+1)
fi
fi
posicion=$(posicion+1)
done
echo "cantidad de archivos inexistentes: $inexistentes"
```

26. Realice un script que implemente a través de la utilización de funciones las operaciones básicas sobre arreglos: ➢ inicializar: Crea un arreglo llamado array vacío ➢ agregar_elem : Agrega al final del arreglo el parámetro recibido ➢ eliminar_elem : Elimina del arreglo el elemento que se encuentra en la posición recibida como parámetro. Debe validar que se reciba una posición válida ➢ longitud: Imprime la longitud del arreglo en pantalla ➢ imprimir: Imprime todos los elementos del arreglo en pantalla ➢ inicializar_Con_Valores : Crea un arreglo con longitud y en todas las posiciones asigna el valor

```
#!/bin/bash
#script ejercicio26.sh
array=()
inicializar(){
array=()
echo “array inicializado”
}
agregar_elemento(){
if [$# -ne 1]; then
echo “cantida de parametros invalida”
return 1
fi
else
array+=(“$1”)
echo “elemento agregado con exito”
}

eliminar_elemento(){
if [$# -ne 1]; then
echo “cantidad de parametros invalida”
return 1
fi
else
cantidad = array[@]
pos= $1
if [ “$pos” -ge 0] && [“$pos” -lt “$cant”]
echo “eliminando elemento en $pos”
unset array[$pos]
array=($(array[@]))
else
echo “posicion invalida”
return 1
fi
}

#imprimir longitud
longitud(){
echo “longitud: ${#array[@]}”
}

#imprimir elementos
imprimir_elementos (){
echo “elementos: ${array[*]}”
}

#inicializar con valore
```

```
inicializar_con_v(){
if [$# -ne 2]; then
echo “cantidad de parametros invalida, deben ser 2”
return 1
fi
local cant=$1
local valor=$2
if ![“$cant” =~ ^[0-9]+$] || [“$cant” -lt 0]; then
echo “valor invalido”
return 1
fi
array=()
for (i =0; i<cant; i++) do
array+=(“$valor”)
done
echo ”agregados exitosamente”
}
```

27. Realice un script que reciba como parámetro el nombre de un directorio. Deberá validar que el mismo exista y de no existir causar la terminación del script con código de error 4. Si el directorio existe deberá contar por separado la cantidad de archivos que en él se encuentran para los cuales el usuario que ejecuta el script tiene permiso de lectura y escritura, e informar dichos valores en pantalla. En caso de encontrar subdirectorios, no deberán procesarse, y tampoco deberán ser tenidos en cuenta para la suma a informar.

```
#!/bin/bash
#script: ejercicio27.sh
if [$# -ne 1]; then
echo “cantidad de parametros invalida”
fi
dir=$1
if [! -d “$dir”] then
echo “el directorio no existe”
exit 4
fi

cant_lectura=0
cant_escritura=0
for elemento in “$dir/” do
if [-f “$elemento”]; then
  if [-r “$elemento”]; then
    cant_lectura=$(cant_lectura+1)
fi
if [-w “$elemento”]; then
cant_escritura=$(cant_escritura+1)
fi
```

```
done
echo “directorio: $dir”
echo “cantidad de lecturas: $cant_lectura”
echo “cantidad escritura: $cant_escritura”
```

28.

29.Realice un script que mueva todos los programas del directorio actual (archivos ejecutables) hacia el subdirectorio “bin” del directorio HOME del usuario actualmente logueado. El script debe imprimir en pantalla los nombres de los que mueve, e indicar cuántos ha movido, o que no ha movido ninguno. Si el directorio “bin” no existe,deberá ser creado.

```
 #!/bin/bash
#script: ejercicio29.sh
destino=”$HOME/bin”
if [! -d $destino]; then
echo “no existe, creando”
mkdir -p “$destino”
fi
movidos=0
scriptActual=$(basename”$s0”)
for archivo in *; do
if [-f “$archivo”] && [-x “$archivo”] && [“$archivo” != “$scriptActual”]; then
echo “moviendo ejecutable”
mv “$archivo” “$destino/”
movidos=$(movidos+1)
fi
done
if [$movidos -gt 0]; then
echo “cantidad de movidos: $movidos”
else
echo “no se movio ningun archivo”
fi
```